APPROACH ABSTRACT

Kavach360

Threat-Model-First AI Cyber Reasoning Platform

What We Plan To Build

Kavach360 is a web platform that studies source code, generates an AI-assisted threat model, gets user verification, runs targeted analysis in isolated containers, correlates evidence into real vulnerability tickets, and validates whether fixes address the root cause.

Threat Modeling | Static Analysis | Fuzzing | LLM Reasoning | Fix Validation

Executive Summary

Kavach360 moves beyond raw scanner alerts. It first understands the system, asks the user to verify its assumptions, and then runs analysis based on the approved model.

01
Threat model before scanning

02
Containerized analysis jobs

03
Evidence-backed issue tickets

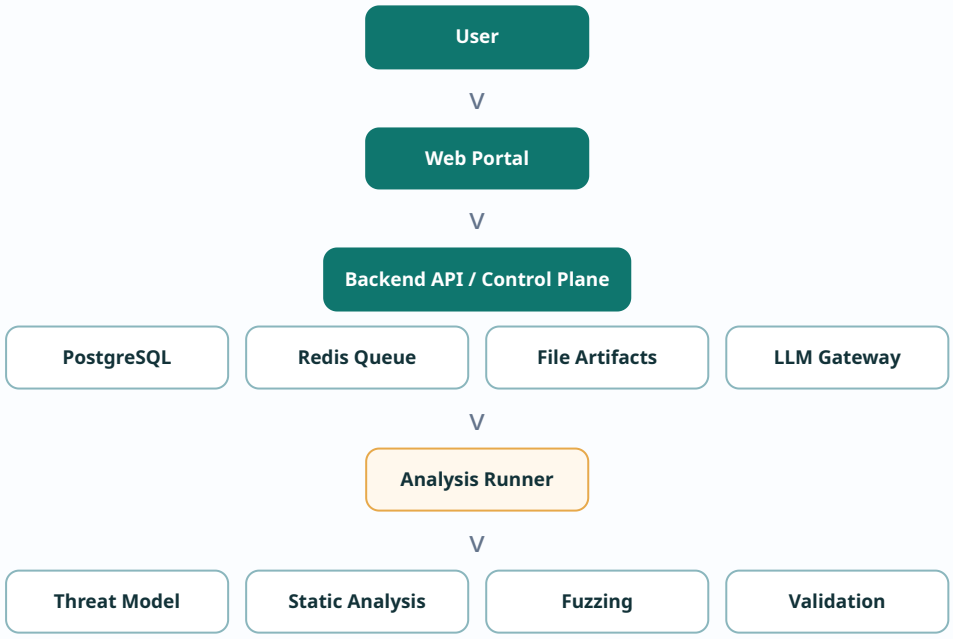
CORE IDEA

The platform ingests source code, project metadata, build details, runtime assumptions, and deployment context. AI generates a structured threat model covering components, entry points, assets, trust boundaries, data flows, external dependencies, and assumptions.

WHY IT MATTERS

Security tools often produce noisy results because they lack architecture context. Kavach360 uses the verified threat model to decide what to scan, fuzz, reason about, prioritize, and validate.

High-Level Architecture



The backend is the control plane. Heavy work runs in isolated Docker containers and writes source snapshots, logs, reports, crashes, diagrams, patches, and validation evidence into a local artifact volume.

Threat-Model-First Pipeline

01

Create project and ingest source code, metadata, build commands, and runtime context.

02

Generate threat model with components, entry points, trust boundaries, assets, and diagrams.

03

User verifies assumptions and corrects inaccurate architecture or build details.

04

Generate an approved analysis plan from the verified threat model.

05

Run static analysis, dependency checks, secret scans, build/test, fuzzing, and LLM review.

06

Correlate raw evidence, reachability, exploitability, and threat-model context.

07

Create vulnerability tickets with evidence, exploit flow, severity, and suggested fix.

08

Validate patches using reproducers, tests, focused scanning, and fuzzing.

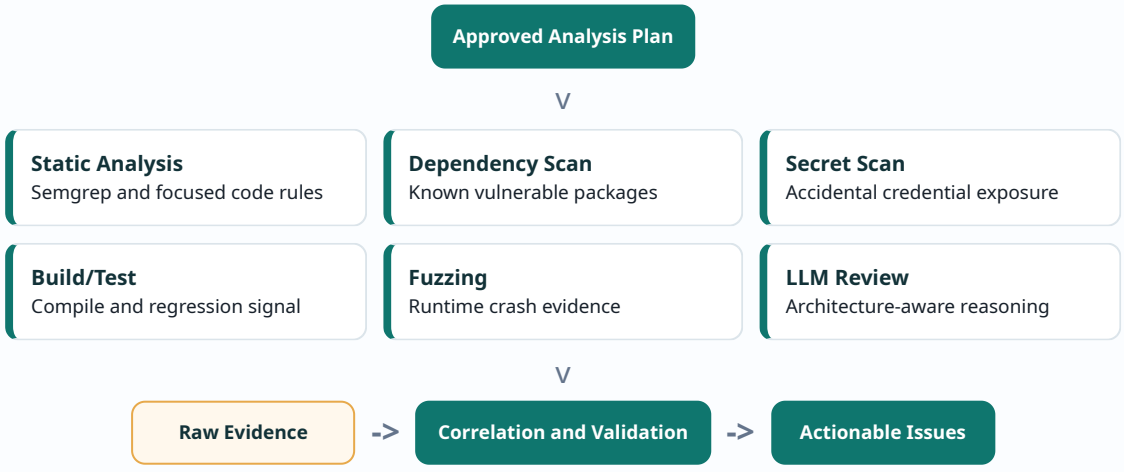
THREAT MODEL OUTPUT

- System overview and component inventory.
- Actors, entry points, assets, data flows, and external services.
- Trust boundaries and security assumptions.
- Architecture and data-flow diagrams.
- Recommended analysis focus areas.

ANALYSIS PLAN OUTPUT

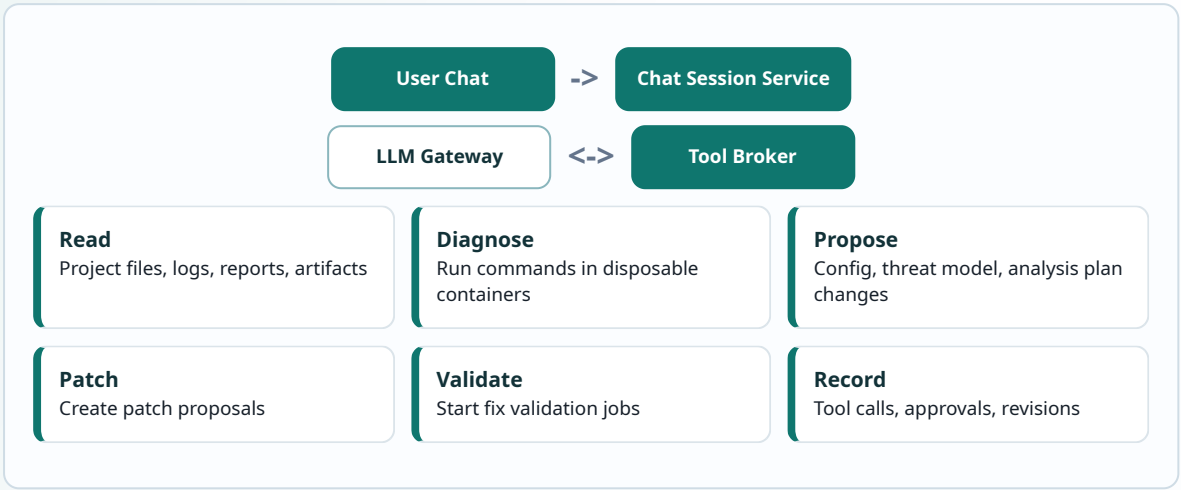
- Selected scanners, fuzzers, skills, and LLM review tasks.
- Build/test commands and working directories.
- Resource limits, timeouts, and container images.
- Scope exclusions and risk-prioritized targets.

Analysis Execution



Integrated AI Chat And Controlled Tools

The AI chat is a contextual operator console. It can help correct wrong assumptions, debug failed jobs, explain findings, propose changes, and start validation workflows.



PERMISSION MODEL

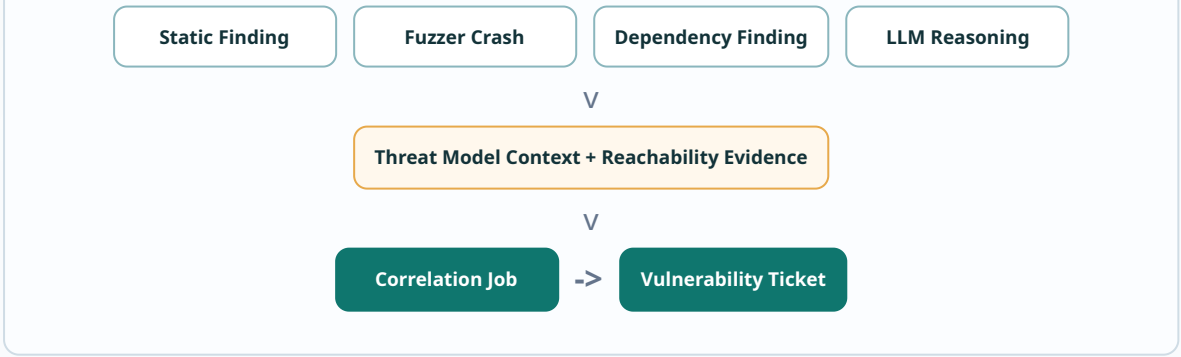
- Read-only project tools can run automatically.
- Diagnostics run only in disposable containers.
- State-changing edits require user approval.
- Threat model and analysis plan changes create revisions.

EXAMPLE CORRECTION

User says: "This project actually builds with ./gradlew build inside services/api."

The chat proposes a build profile update, asks for approval, runs a build check in an isolated container, and records the corrected configuration.

Finding Correlation To Issues



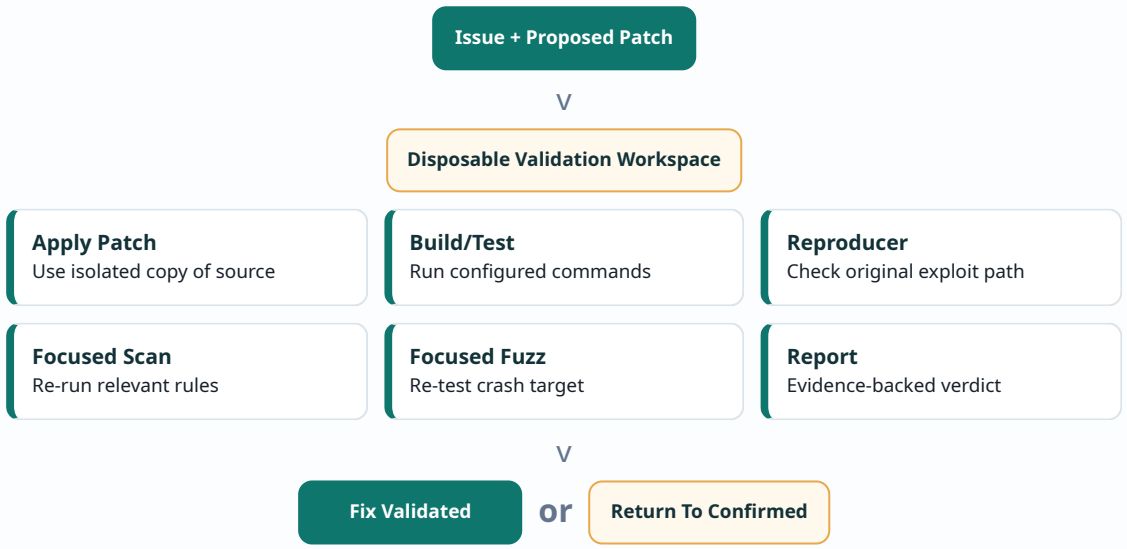
Issue Tickets And Fix Validation

ISSUE TICKET INCLUDES

- Title, severity, CWE, status, confidence, and exploitability.
- Affected component, files, functions, entry point, and trust boundary.
- Impact, evidence, reproduction steps, exploit scenario, and artifacts.
- Suggested fix, patch proposal, validation history, and flow diagram.

ISSUE LIFECYCLE

- Open -> Triaged -> Confirmed.
- Confirmed -> Fix Proposed -> Fix Under Validation.
- Fix Under Validation -> Fix Validated -> Resolved.
- Rejected or reopened when evidence does not hold.



Data And Storage

Layer	Purpose
PostgreSQL	Projects, source snapshots, build profiles, threat models, analysis plans, runs, jobs, findings, issues, chat sessions, approvals, skill runs, and model configs.
Redis	Async queue for threat modeling, analysis, correlation, diagram generation, and validation jobs.
Local Artifact Store	Source snapshots, logs, reports, crashes, coverage, diagrams, patches, and validation evidence under /artifacts.
Skills	Versioned recipes that define inputs, tools, prompts, containers, outputs, and validation criteria.

Planned Tech Stack

Next.js | React | TypeScript | Tailwind CSS | FastAPI | PostgreSQL | Redis | Docker Compose | Semgrep | libFuzzer/AFL++ | Mermaid.js | OpenAI | Claude | Gemini | Ollama/vLLM

INITIAL SCOPE

- Docker Compose deployment.
- Web portal and backend API.
- PostgreSQL, Redis, and local artifact storage.
- Threat model generation and review.
- Analysis planning, static analysis, fuzzing, and correlation.
- Issue creation, patch proposal, and fix validation.
- Integrated AI chat with controlled tools.
- Provider-neutral LLM configuration.

DEFERRED SCOPE

- Keycloak/OIDC authentication.
- Multi-tenant RBAC.
- Kubernetes deployment.
- MinIO/S3 artifact storage.
- Distributed fuzzing cluster.
- Enterprise audit and compliance reporting.

Security Design

Analysis jobs may execute untrusted code, so the platform isolates execution from the web/API service. Jobs run in separate containers with scoped mounts, resource limits, non-root users where possible, and network access disabled by default.

The API should not mount the Docker socket. The runner is the only service responsible for launching job containers. Secrets should be protected, and source context sent to external LLM providers should go through redaction controls.

Every important AI action is recorded as a tool call, approval, revision, artifact, or skill run. This makes the system reproducible and easier to audit.

Kavach360 is intentionally modular: the first version remains achievable with Docker Compose and local storage, while the architecture can later evolve toward Kubernetes, distributed analysis, and object storage.